



CSC207 Software Design



An introduction to software design and development concepts, methods, and tools using a statically-typed object-oriented programming language such as Java. Topics from: version control, unit testing, refactoring, object-oriented design and development, design patterns, advanced IDE usage, regular expressions, and reflection.



Volunteer Notetaker Needed In This Class!

Sign up today

- We are looking for dedicated students to sign up as a notetaker in this course.
- Support your peers who are registered with Accessibility Services by volunteering as a notetaker.
- Upon completion of volunteering, notetakers are eligible to receive a certificate of appreciation, and add the opportunity to your Co-Curricular Record (CCR)
- Sign up by following the link below or the QR code



For more information, please check out <https://uoft.me/notetaking>.

Welcome to CSC207!

LEC 0101 and 0102, your instructor, Lindsey Shorser (l.shorser@utoronto.ca) wants to discuss AI with you next class.

Please answer and submit an answer to AT LEAST ONE of these anonymously.

The Good: In what ways have you or others used AI effectively?

The Bad: In what ways is AI having a negative effect on you, others, or your world?

The Complex: Is there something about your relationship to AI that you want us to know going into this course?

Instructor intro: Jonathan Calver

- BSc CS and Physics @ USask
 - contact tracing and spread of disease
- MSc & PhD in CS @ UToronto
 - ODEs and parameter estimation (numerical & scientific computing)
- Teaching Stream Faculty
 - CS Pedagogy (E.g., [Memory Lab](#), [CS Connections](#))
 - Teaching: 148, 207, 336
- Course Coordinator for CSC207 this term



Instructor intro: Jack Sun

- Teaching Stream Faculty
 - Assistant Professor
 - Teaching: 108, 207, 209, 309, 369, 469
- Website
 - <https://www.cs.toronto.edu/~sunk/>
- Research Interests
 - Systems (e.g., storage, distributed systems)
 - Programming languages
 - CS Education
- Hobbies: Anime and video games 🐱
- Co-instructor for CSC207 this term



Setting the course context

Programming teams grow large, complex applications

- Where do application ideas come from?
- What other roles are involved in designing and creating an application?
- How do programmers discuss the code and what abstractions do they use?
- What tools do they use?
- How do they distribute the work?
- How do they share the code?



Course logistics (see Quercus for details)

Mark Breakdown

- team project [20%]
- in-class team activities during lecture [8%]
- solo homework [11%]
- midterms [20%]
- final exam [40%]
- reflection [0.5%]
- research study [0.5%]

Team Project

- Design and implement a Java Desktop application
- Presentation at the end of term
- Individual mark based on overall team performance and your contribution to the project
- Team formation will be done in lab; you will have input on this process along with your TA in lab
 - Most teams will have 4 members; your TA will help you to form teams based on project interest and other factors
 - Get to know potential team members during the first two weeks of the term!

Midterm Tests

- 2-stage tests
 - Individual test (60 minutes)
 - Break; setting up for the team test (5-10 minutes)
 - Team test (30 minutes; with your project team)
- Out of class:
 - Friday, October 2nd, 5:10–6:50pm
 - Friday, November 13th, 7:10–8:50pm
- If you have an academic time conflict, please email the course address as soon as possible to discuss alternative midterm options

Course Topics and Tools

Topics

Version Control

Object-oriented Programming (OOP)

Design Principles

Clean Architecture

Design Patterns

Ethics

Regex

Tools

Java

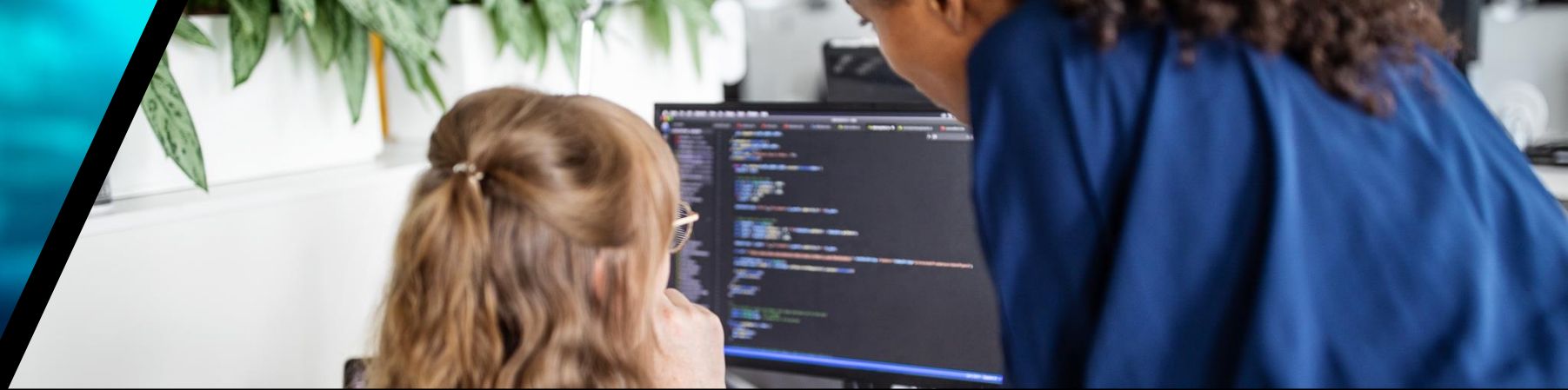
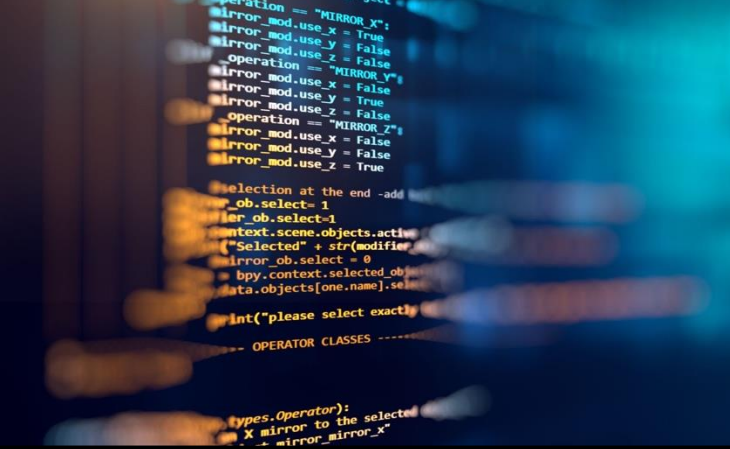
Maven

IntelliJ

Git

GitHub

APIs

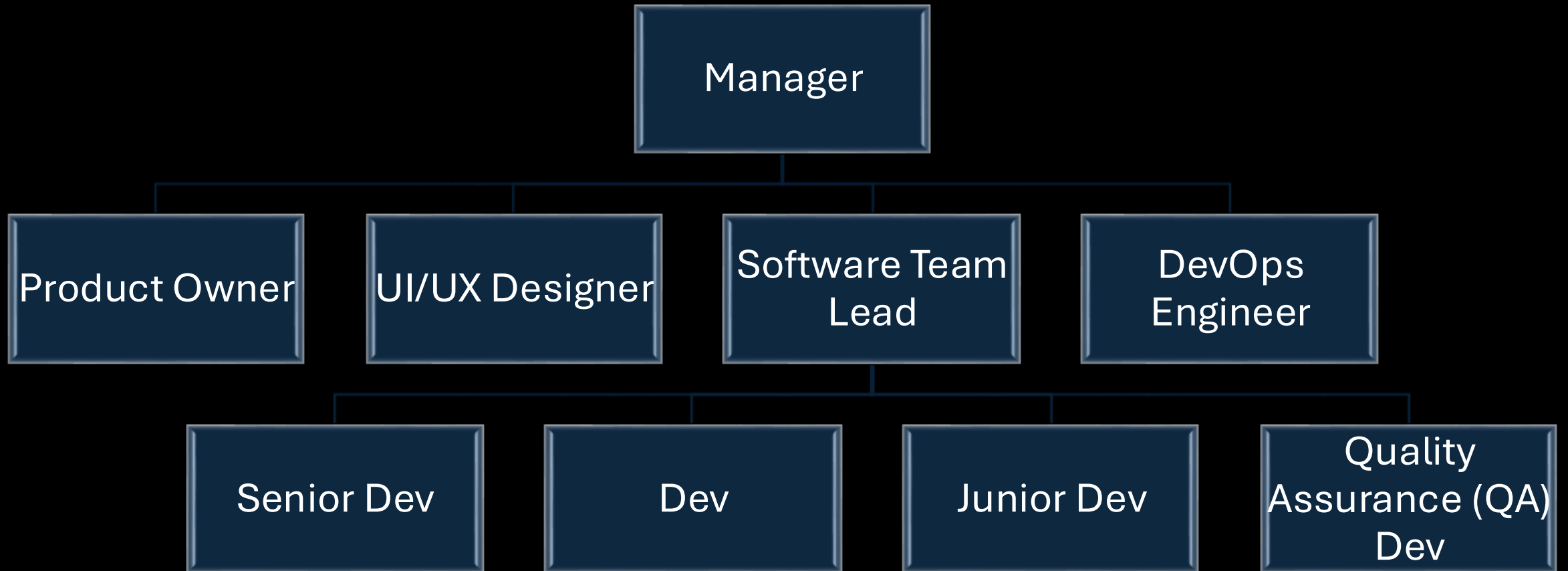


A tour of software design

CSC 207 Software design



Org chart of typical software roles

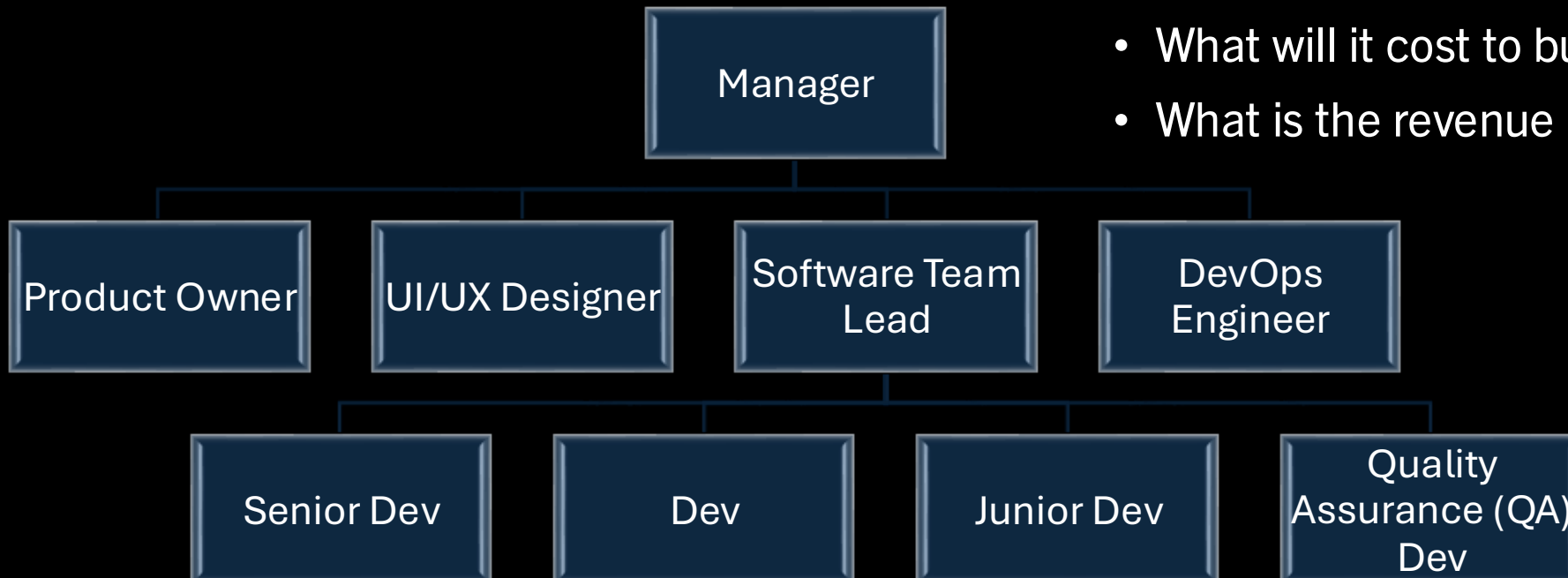


The software lifecycle

1. Planning and Analysis

- Product Owner, Client, Manager
- Goal: evaluate needs of end users, feasibility of project, revenue potential

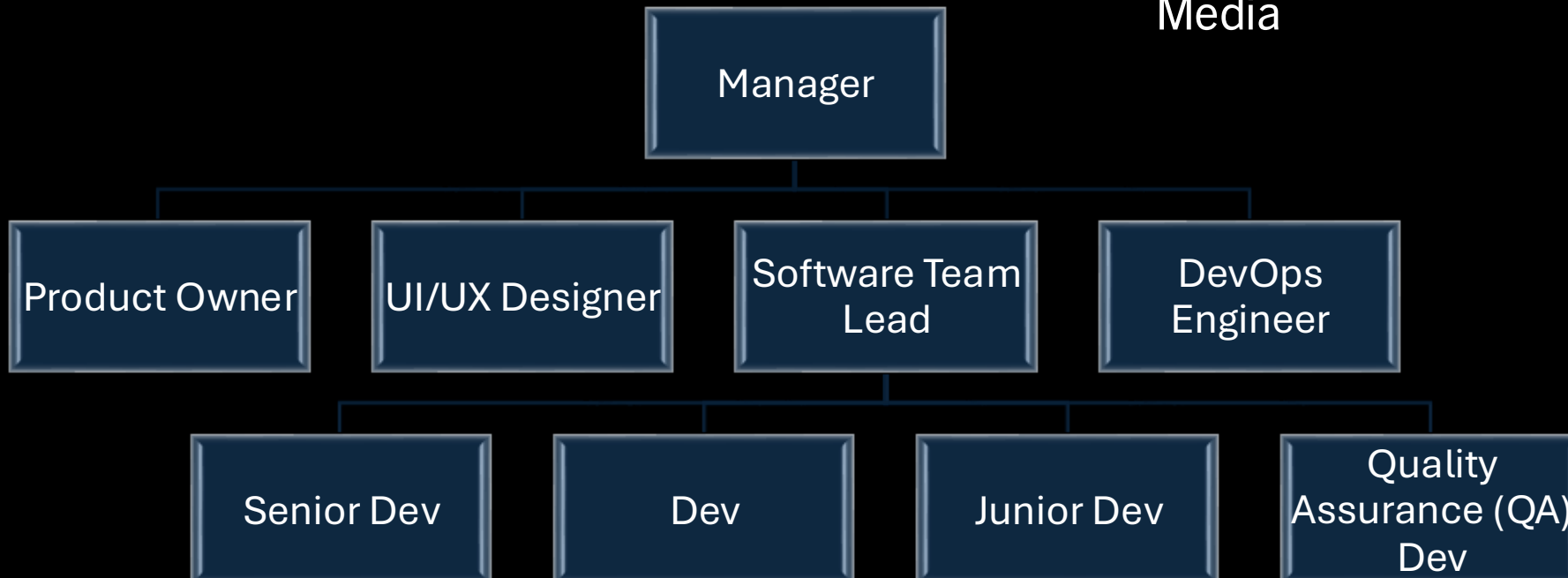
- What does the client want built and what problems will it solve?
- What features will the end users need from the product?
- Roughly how long will it take to build?
- What will it cost to build it?
- What is the revenue potential?



The software lifecycle

2. Define requirements

- Product Owner, Client, UI/UX Designer
- Goal: create clear requirements for the dev team
- Document the use cases for the software
- What will the end users want to do?
 - Make a list of specific tasks and acceptance criteria so you know when you're done
- CSC318 Design of Interactive Computational Media



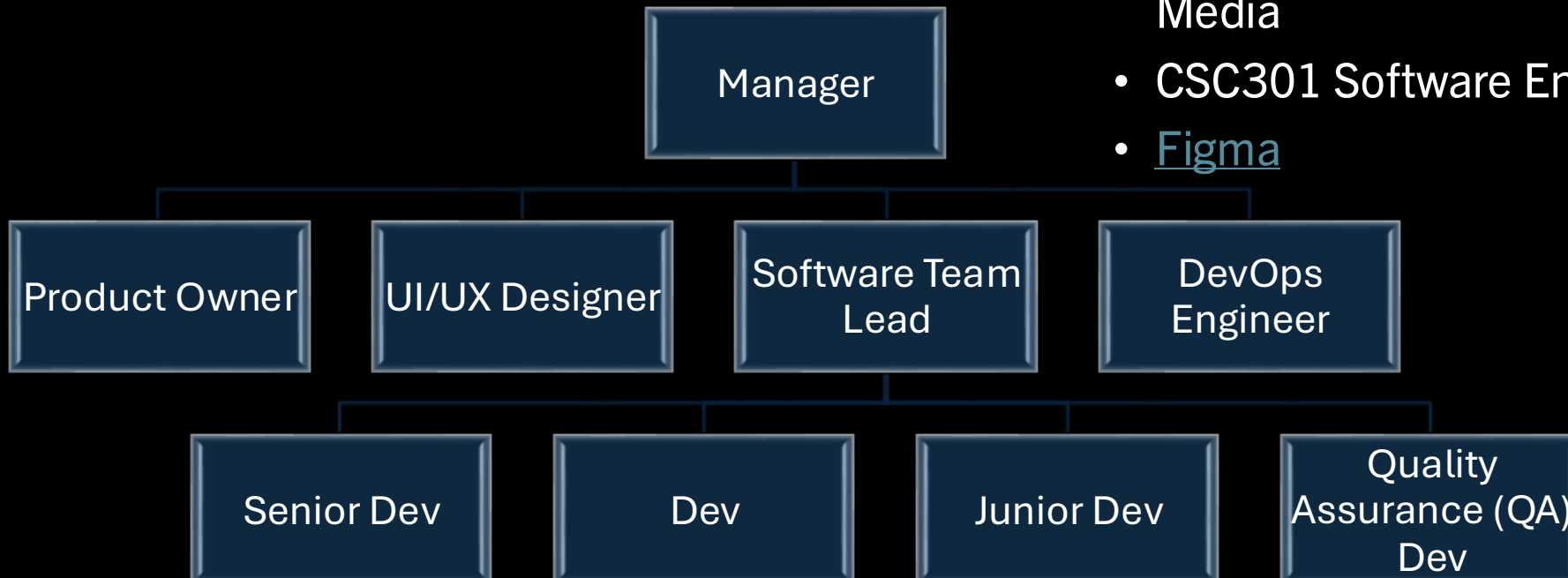
1. Planning and analysis
2. Define requirements

The software lifecycle

3. Design the UI and UX

- Product Owner, Client, UI/UX Designer
- Goal: develop a high-fidelity prototype

- No programming, just design
- Draw detailed pictures to capture what the screens will look like
- Validate prototype with customer
- CSC318 Design of Interactive Computational Media
- CSC301 Software Engineering
- [Figma](#)



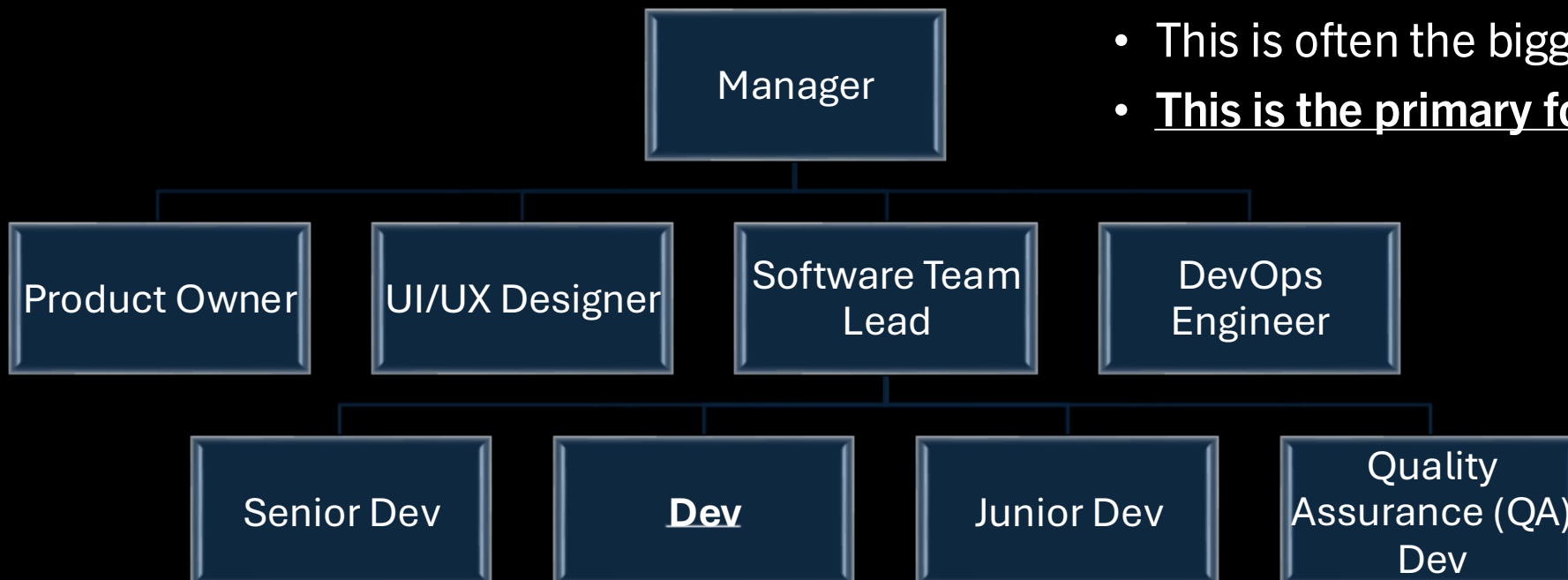
The software lifecycle

1. Planning and analysis
2. Define requirements
3. Design the UI and UX

4. Design and develop the code

- Software Team Lead, Devs, UI/UX Designer
- Goal: develop the actual product

- Design and grow a program that looks and behaves like the prototype
- Apply fancy techniques you'll learn in CSC207 to make it
 - Maintainable
 - Testable
- This is often the biggest part of the work
- **This is the primary focus of CSC207**



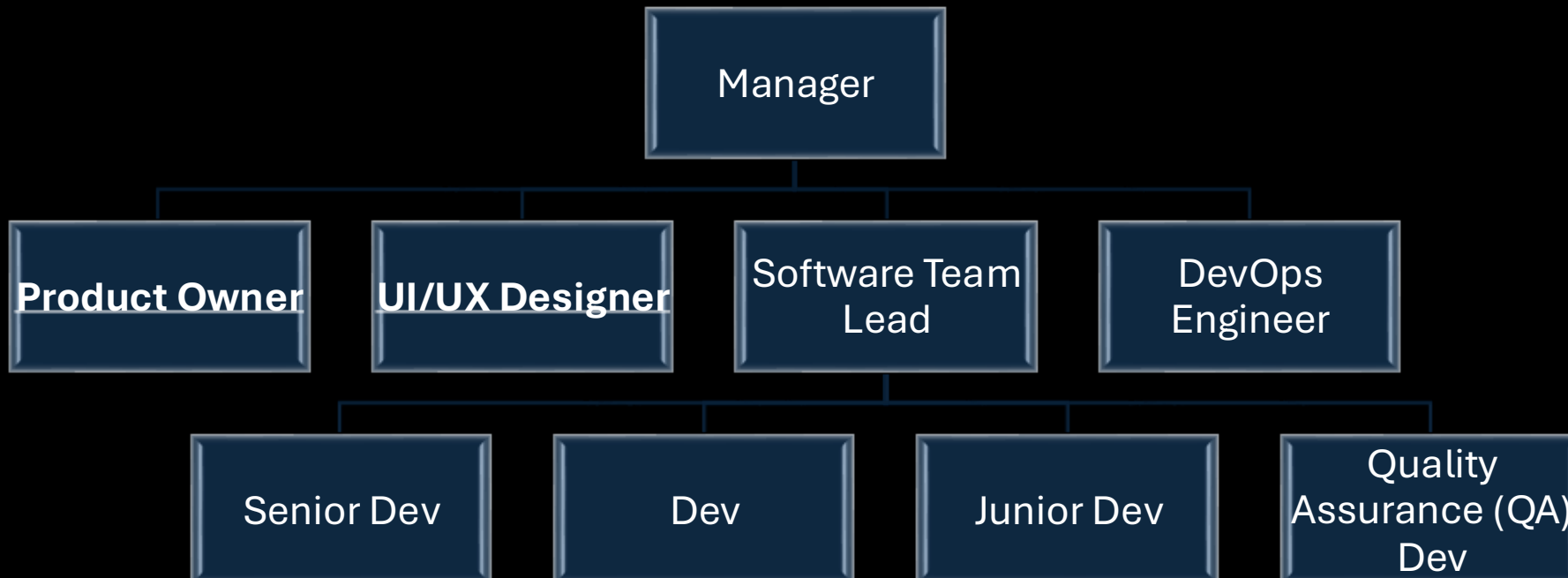
The software lifecycle

1. Planning and analysis
2. Define requirements
3. Design the UI and UX
4. Design and develop code

5. Testing

- QA Dev, Product Owner, DevOps Engineer, Client, Devs
- Goal: make sure it works

- Types of testing:
 - performance, unit, integration, security, usability, acceptance
- CSC207 will focus on unit testing
- CSC301 Software Engineering



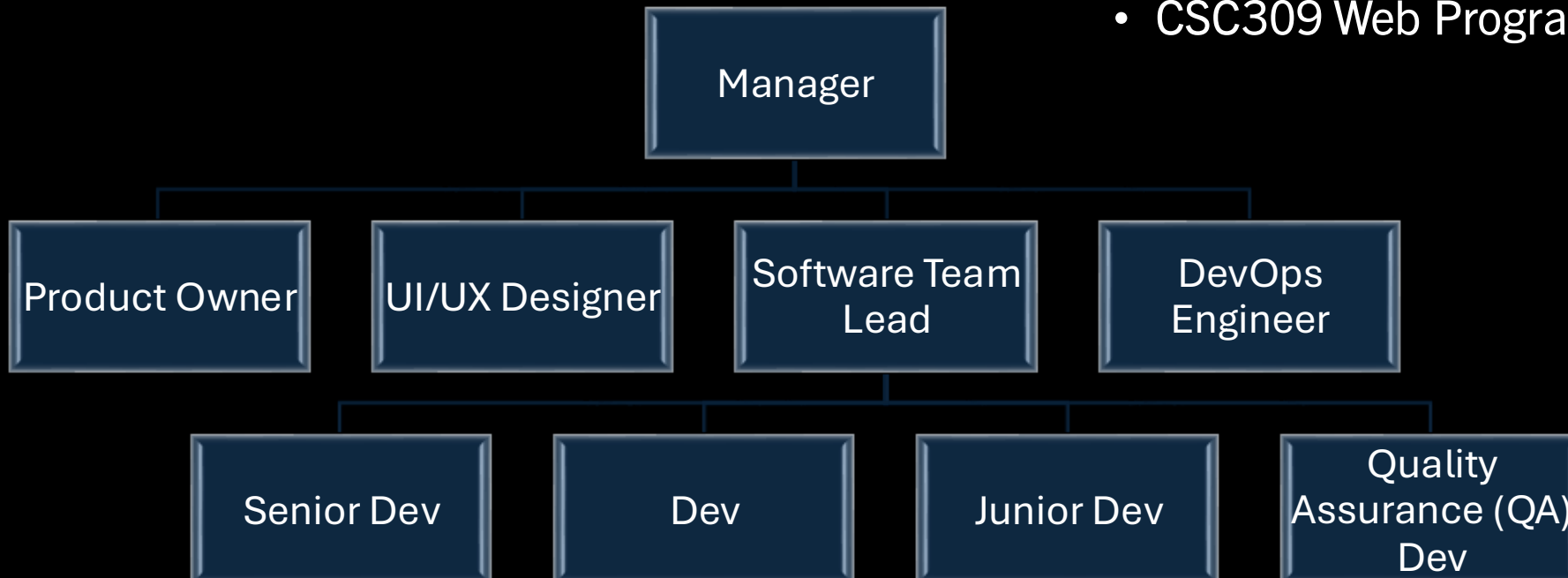
The software lifecycle

1. Planning and analysis
2. Define requirements
3. Design the UI and UX
4. Design and develop code
5. Test

6. Deployment

- DevOps Engineer, Devs, Product Owner, Client
- Goal: deliver it to end users

- Publish in app store
- Create and send an executable
- Update a website
- CSC301 Software Engineering
- CSC309 Web Programming

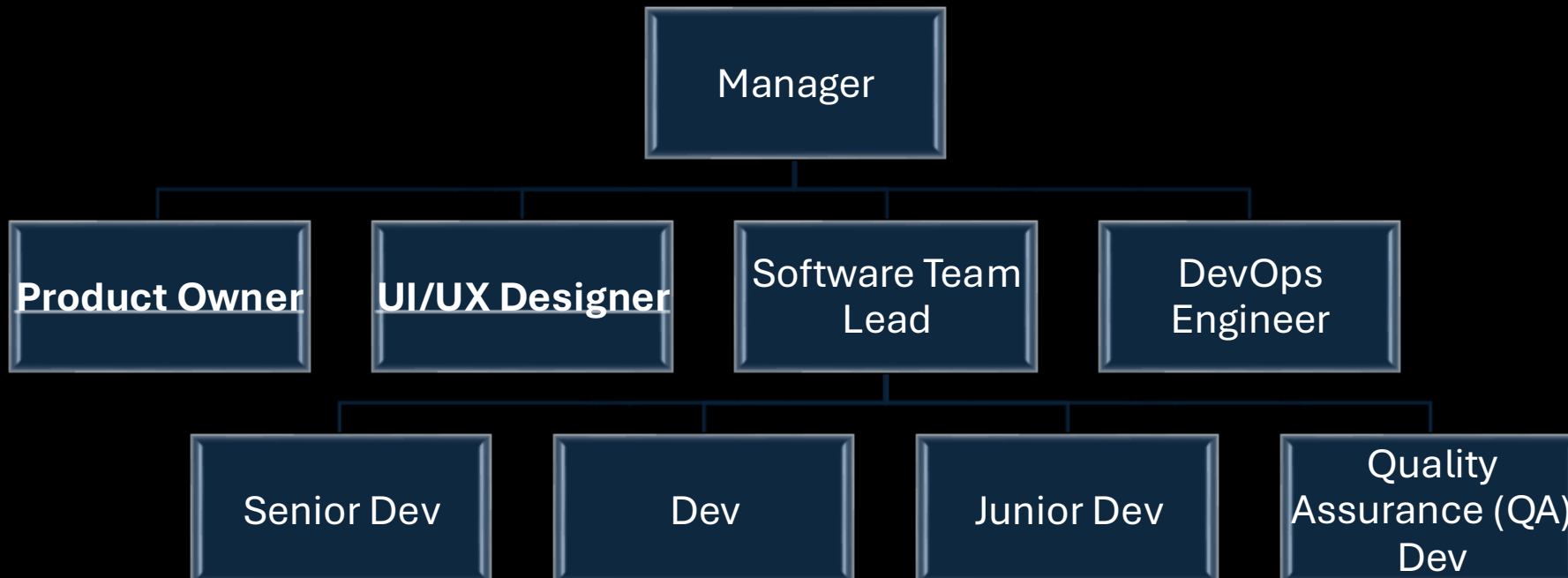


The software lifecycle

1. Planning and analysis
2. Define requirements
3. Design the UI and UX
4. Design and develop code
5. Test
6. Deploy

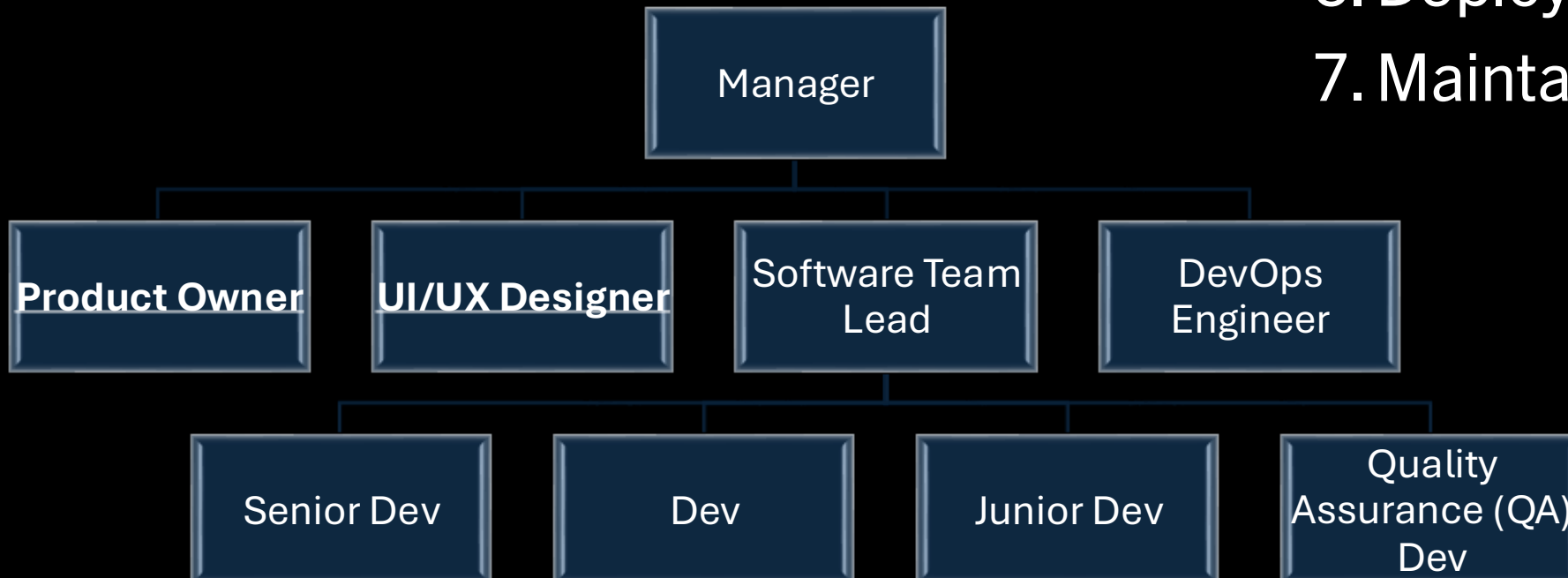
7. Maintenance

- DevOps Engineer, Devs, Product Owner, Client
- Goal: keep it going
- Track and address bugs
- Ensure client needs are being met
- Plan for new features

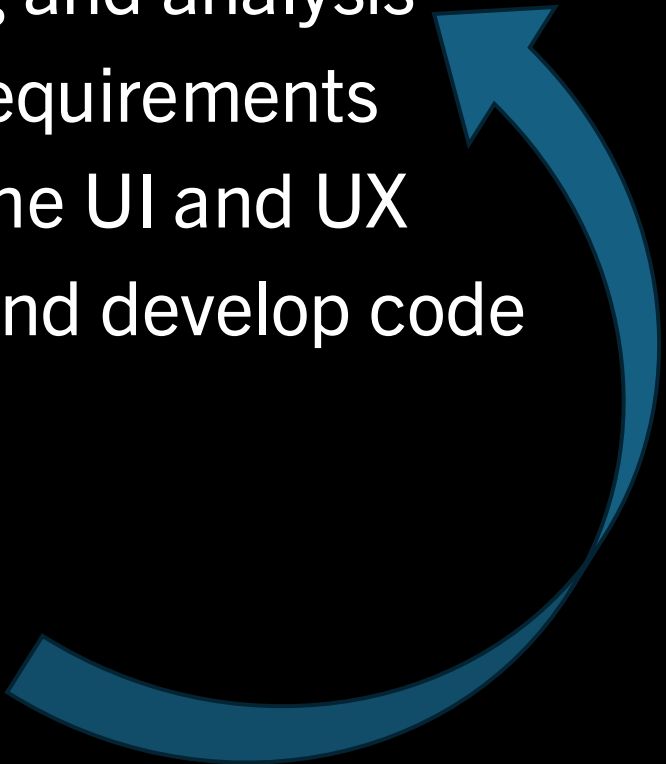


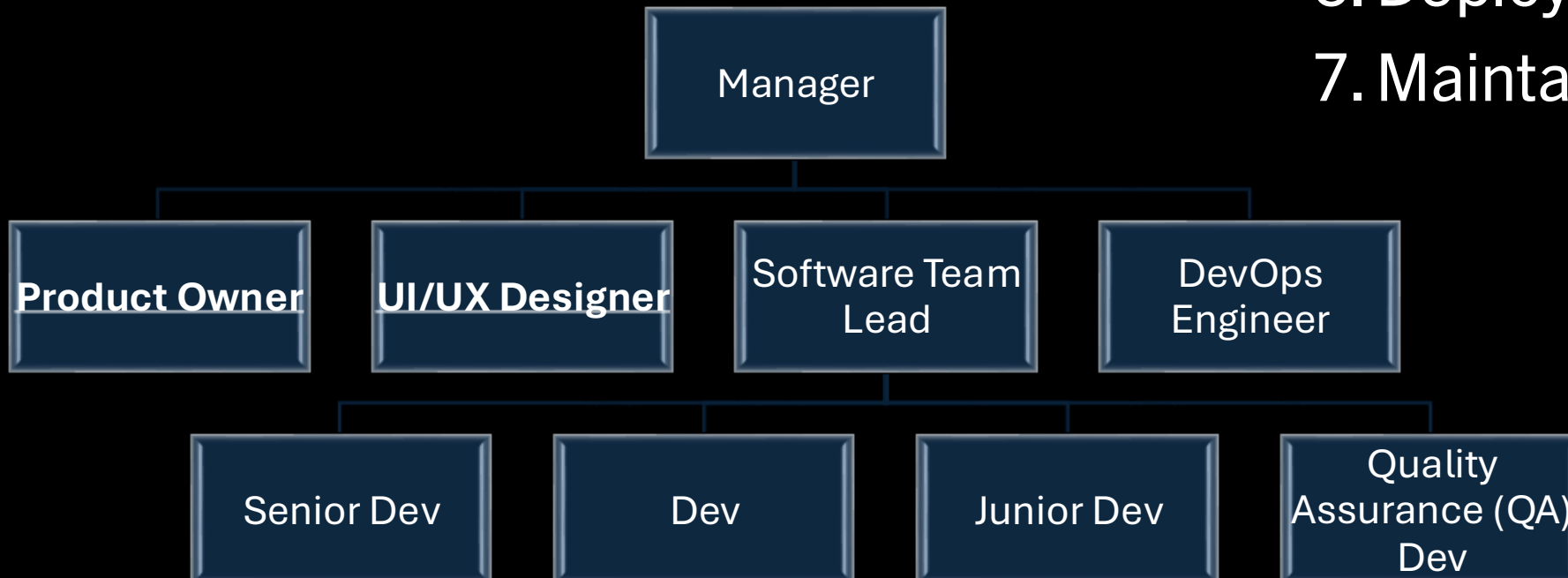
The software lifecycle

1. Planning and analysis
2. Define requirements
3. Design the UI and UX
4. Design and develop code
5. Test
6. Deploy
7. Maintain



The software lifecycle *is iterative!*

1. Planning and analysis
 2. Define requirements
 3. Design the UI and UX
 4. Design and develop code
 5. Test
 6. Deploy
 7. Maintain
- 



Communicating about software

How do we communicate
about software systems?

Discussion Activity

Think of one software project that you have
worked on, like a course assignment.

Try to explain what you built to someone around
you — only speaking for 30 seconds.

Communicating about software

How do we communicate
about software systems?

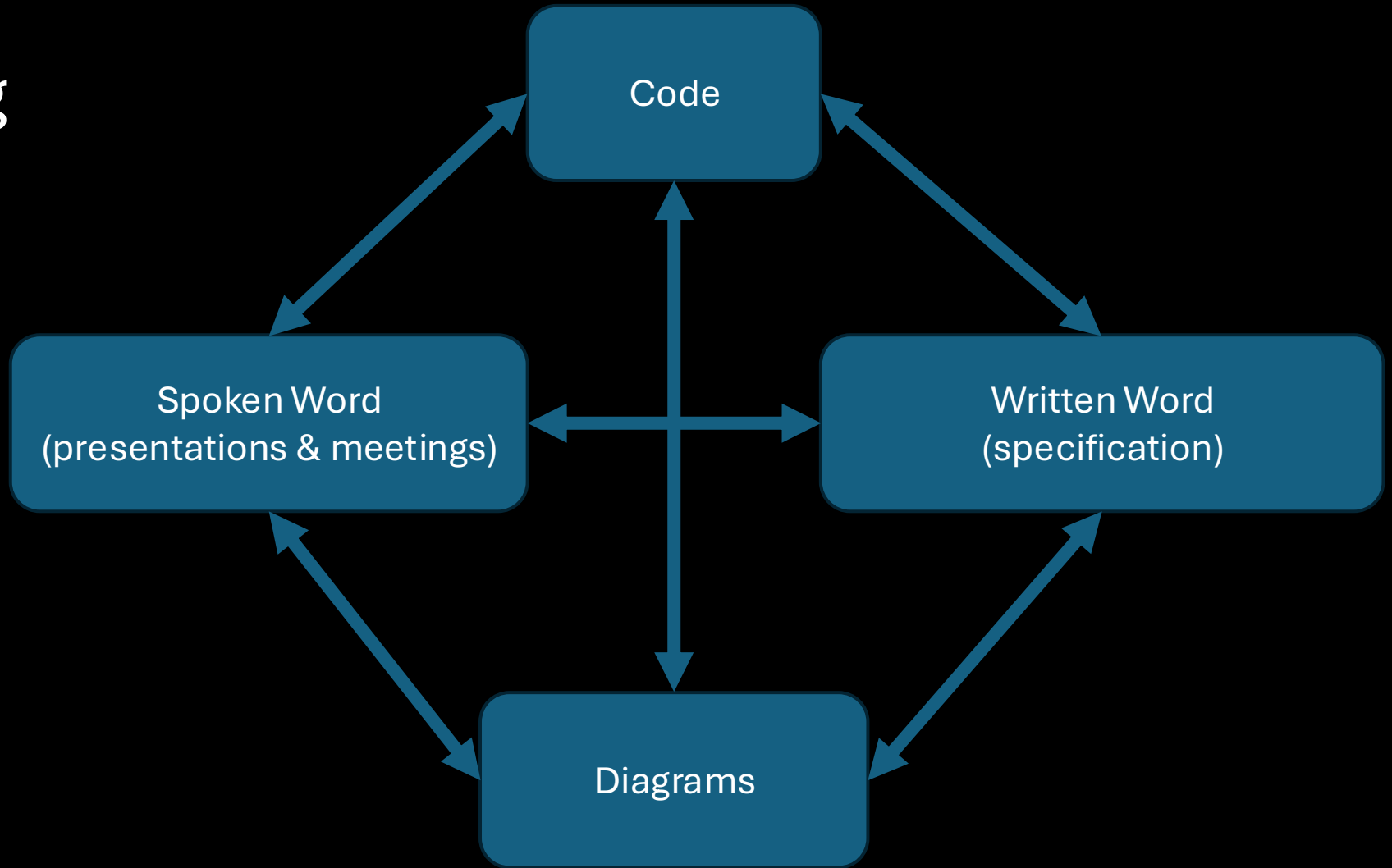
Discussion Activity Debrief

Was it easy to explain your project?

What would have made it easier to
communicate?

Communicating about software

How do we communicate about software systems?



Team Based Learning

TBL encourages:

- Development of professionalism and communication skills
- Community building and sense of belonging

According to the literature on TBL, if you stay present and participate in class, you should expect:



To learn and remember more



To understand the material better



To master transferable skills that will help you in industry

Helping each other learn will benefit the team!

GenAI in software development

- Give an example where you've had a good experience using generative AI to assist you during software development.
- Give an example where you've had a bad experience using generative AI to assist you during software development.
- In the software development life cycle, who is responsible for reviewing code produced by generative AI?

Upcoming In

CS



Find more
on our
Quercus



Computer Science
UNIVERSITY OF TORONTO

Sept 17, 5 -7pm

AI Industry Panel

Join us for a panel discussion with industry experts sharing their perspectives on emerging opportunities, and the ways AI is transforming research and industry



Register here!

[illegible]

CSC108/148/110/111
stuff you know

CSC108/148/110/111 stuff you know

value and type;
expressions

naming a value using
an assignment
statement (assigning a
value to a variable)

control flow: sequence
of statements, if, while,
for, function call, return
statement, call stack,
recursion

ADTs and data
structures: string, list,
dictionary, linked list,
stack, queue, tree

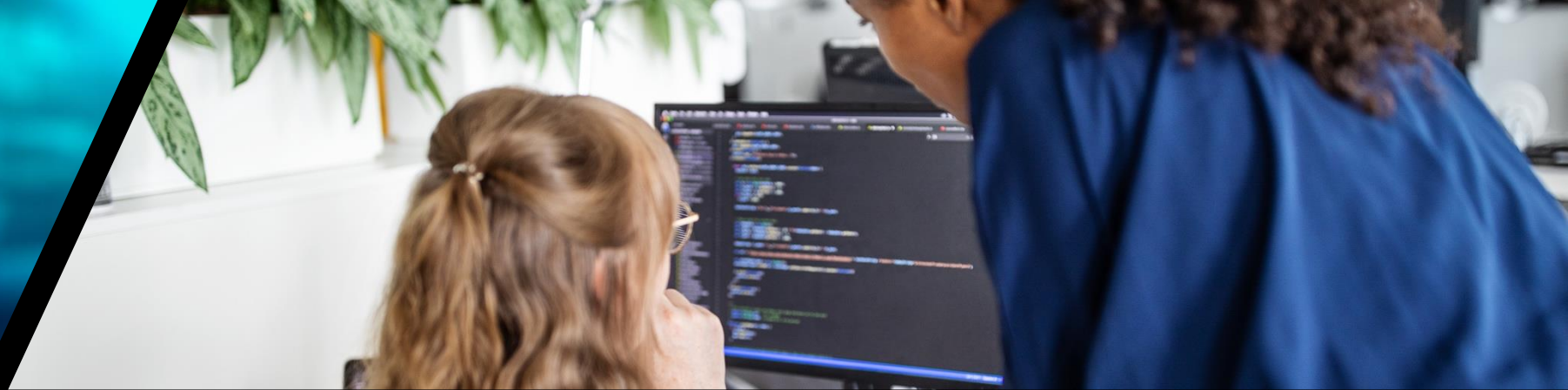
classes and the
objects they describe;
composition;
inheritance (OOP)

some variables and
methods are private
(Python: use a leading
_underscore)

computational
complexity (big-Oh)

unit testing, debugging

function and class
design recipes —
processes by which to
write code



Intro to Java

CSC 207 Software design



Reference Material on Java

- The "course notes" and practice Java quizzes on Quercus:
 - <https://github.com/CSC207-UofT/207-course-notes/blob/master/01-introduction-to-java.md>
 - https://q.utoronto.ca/courses/353377/pages/java-practice-quizzes?module_item_id=6076633
- The official java tutorials are also a great place to learn:
 - <http://docs.oracle.com/javase/tutorial/java/TOC.html>
- An optional mini e-textbook:
 - <https://runestone.academy/ns/books/published/java4python/index.html>
- Ask on Piazza if you have questions or visit office hours. Everyone is here to help.
- [LearnAid](#) will be available as an additional resource; see details on Quercus.

Version of Java in this course

- In most materials we focus on the core language features and point to the official Java tutorials, which were written for **Java 8**.
- There are some nice language features in later versions of Java, but we largely won't be emphasizing them in this course. E.g., the Java 10 `var` keyword; records in Java 16.
- You will only be tested on the core language features.
- Once you start working on the project, your team can agree on which version of Java you want to use.

Running Programs

What is a program?

What does it mean to “run” a program?

To run a program, it must be translated from its high-level programming language to a low-level machine language whose instructions can be executed.

Roughly, there are two flavours of translation:

- **Interpretation**
- **Compilation**

Interpreted vs. Compiled

Interpreted (e.g., Python)

- Python is a program (an interpreter) that translates and executes one statement of a program at a time.

Compiled (e.g., C)

- A C compiler translates an entire C program to an executable file. The translated program contains machine code that runs natively on an operating system.

Hybrid (e.g., Java)

- Java programs are compiled to bytecode (which is a lot like assembly code) using the Java compiler. This is an intermediate representation of the program.
- The Java Virtual Machine (JVM) is an executable that runs this intermediate bytecode, translating and executing one statement of the bytecode program at a time.

Compiling Java

- You need to compile, then run (as described above).
- If using the command line, you need to do this *manually*. For example:

```
javac HelloWorld.java // Compile using javac to produce file “HelloWord.class”
```

```
java HelloWorld // Run the program using java
```

- Modern IDEs (like IntelliJ) do this for you with the help of a build system (e.g. maven or gradle)
 - Build systems take care of the build process, so that you can focus on the code!
- But you should have some idea of what’s happening under the hood!
 - If you go on to take CSC209, you will learn C and more about compiling code.

Computer Architecture

- 108/148 and 110/111: we focused on applications.
- The operating system (OS) manages the various running applications and helps them interact with the hardware (**CSC209H**, **CSC369H**).
- The OS works directly with the hardware (**CSC258H**, **CSC369H**).

Applications

Operating System

Hardware

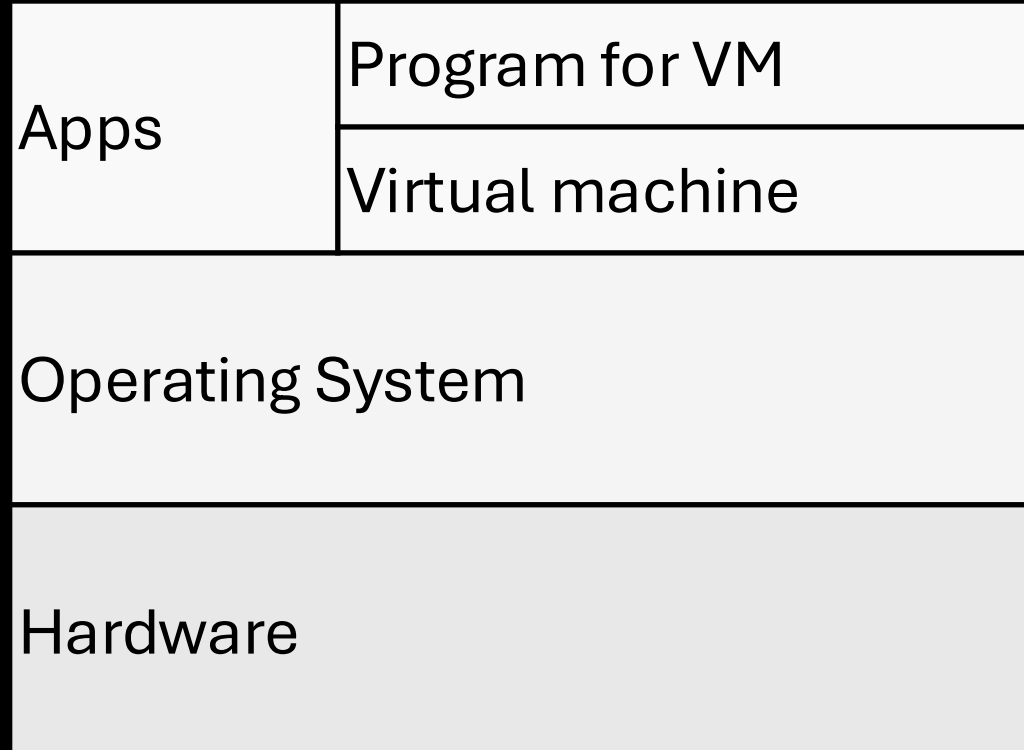
Compiled Applications

- Programs written in languages like C, Objective C, C++, Pascal, and Fortran are **compiled** into OS-specific applications.
- Compilation involves turning human-readable programs into OS-specific machine-readable code (CSC258H, CSC369H, CSC488H).

Apps	Web browsers, Email client, MS Office, etc.
Operating System	
Hardware	

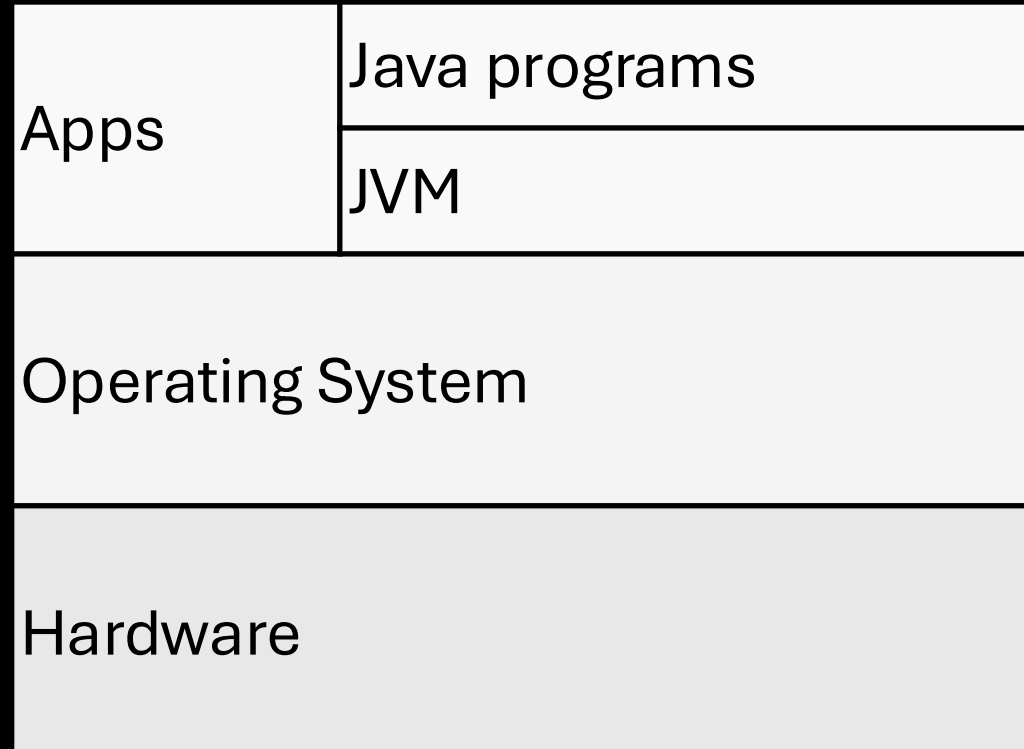
Virtual Machine Architecture

- **Virtual machine (VM):** an application that pretends to be a computer.
- For example, each of Java, Python, and Scheme have their own VM.
- The VM application is written for each OS so that programs are portable across operating systems.



Java Architecture

- **JVM**: an application that is something like an operating system
- Java programs are compiled to an intermediate state called **byte code**: machine code for the JVM
- The same compiled Java program can run in any JVM on any OS!



- JVMs optimize the byte code as it runs (CSC324H, CSC488H) and can even be as fast as code written in C!

Some Notable Aspects of Java

types

primitives

casting

generics

exceptions

interfaces

overloading

memory model

Junit

Java Swing

Javadoc

constructors

access modifiers

packages

Java is a strongly typed language

Unlike Python, Java *requires* that we explicitly declare the type of every variable in our program. I.e., type annotations are mandatory in Java for your code to even compile!

`x = "hello";` // fine in Python, but not in Java!

`int x = 5;`

Java is a strongly typed language

Generics and Collections

lst: list[int] = [1, 2, 3] // a list of integers in Python

List<Integer> lst = ...;

d: dict[str, str] = {'[' : ']', '<' : '>'} // a dictionary of str -> str in Python

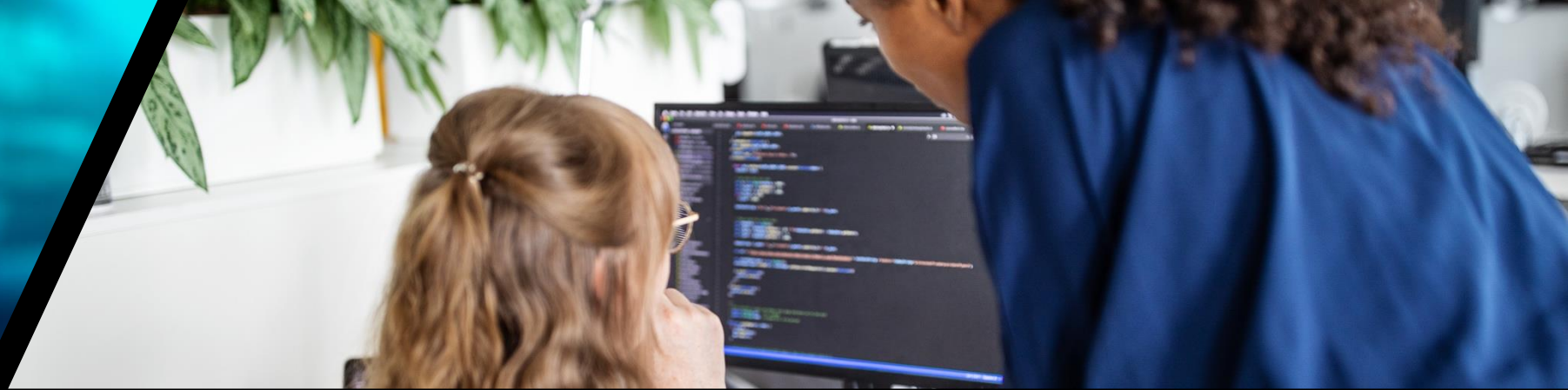
Map<String, String> map = ...;

Java is an object-oriented language

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Every program requires a static main method.

This differs from Python, where you can just write a script.



Version Control

CSC 207 Software design



Motivation

Group projects often involve some kind of shared deliverable.

How do you currently collaborate when working on such a group project? (And does this work well, in your experience?)

What is Version Control?

- Software development teams need a way to track how their software changes over time.
- A main repository (repo) of files exists on a server.
- People “clone” the repo to get their own local copy.
- As significant progress is made, people “push” their changes to the main repo, and “pull” other people’s changes from the main repo.
- The repo **keeps track of every change**, and people can revert to older versions.

Further reading:

<https://www.atlassian.com/git/tutorials/what-is-version-control>

Why Version Control?

Why Version Control?

- **Backup and restore** - because accidents happen
- **Synchronization** - multiple people can make changes
- **Short term undo** - that last change made things worse?
- **Long term undo** - find out when a bug was introduced
- **Track changes** - all changes related to a bug fix
- **Sandboxing** - try something out without messing up the main code
- **Branching and merging** - (better defined sandboxes)

Plain text formats

- Plain text formats (Java code, python code, markdown, LaTeX, any other "human-readable" file formats) are well suited to version control.
- Since changes are tracked, one can easily look at the difference (diff) between two versions of the file being tracked.
- Formats like .docx or .xlsx are less meaningfully stored in a version control system, since the differences when you change them don't have any human-interpretable meaning. For those formats, the software developers introduce features like "track changes mode".

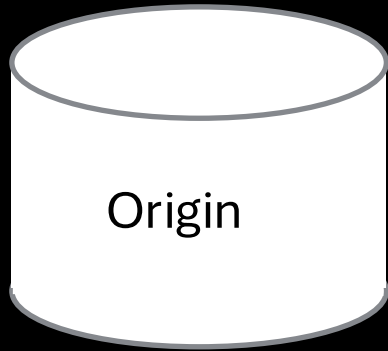
Git

- For CSC207, the remote repository will usually exist on GitHub.
- You will **clone** your remote repository, work on files locally, **add** and **commit** changes to your local repository, and **push** your changes to the remote repository.
- We'll only teach the basics in this course, so that you know enough to use it in your group project. (learn as you go)

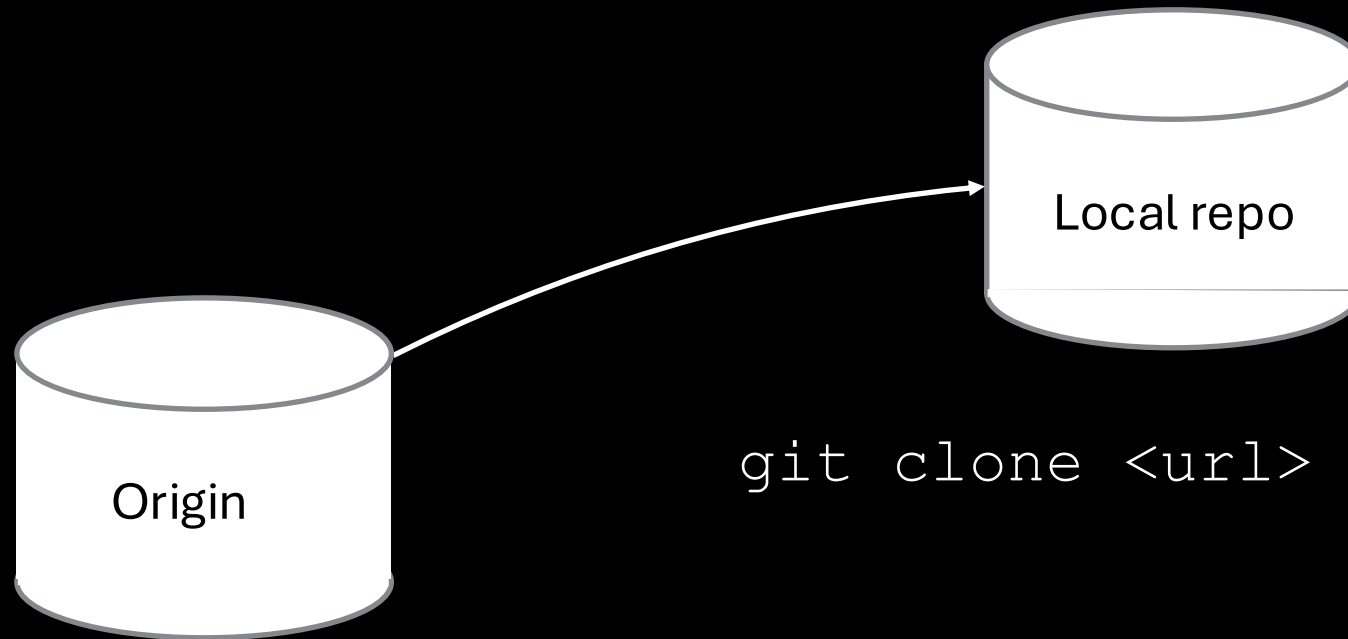
Further reading: In the spirit of thinking about software design, we encourage you to read <https://en.wikipedia.org/wiki/Git>, which discusses the motivation for why Linus Torvalds decided to create git, as well as the system's design.

Remote repository

- This is the repo that lives on another server.
- By convention we call it the *origin*



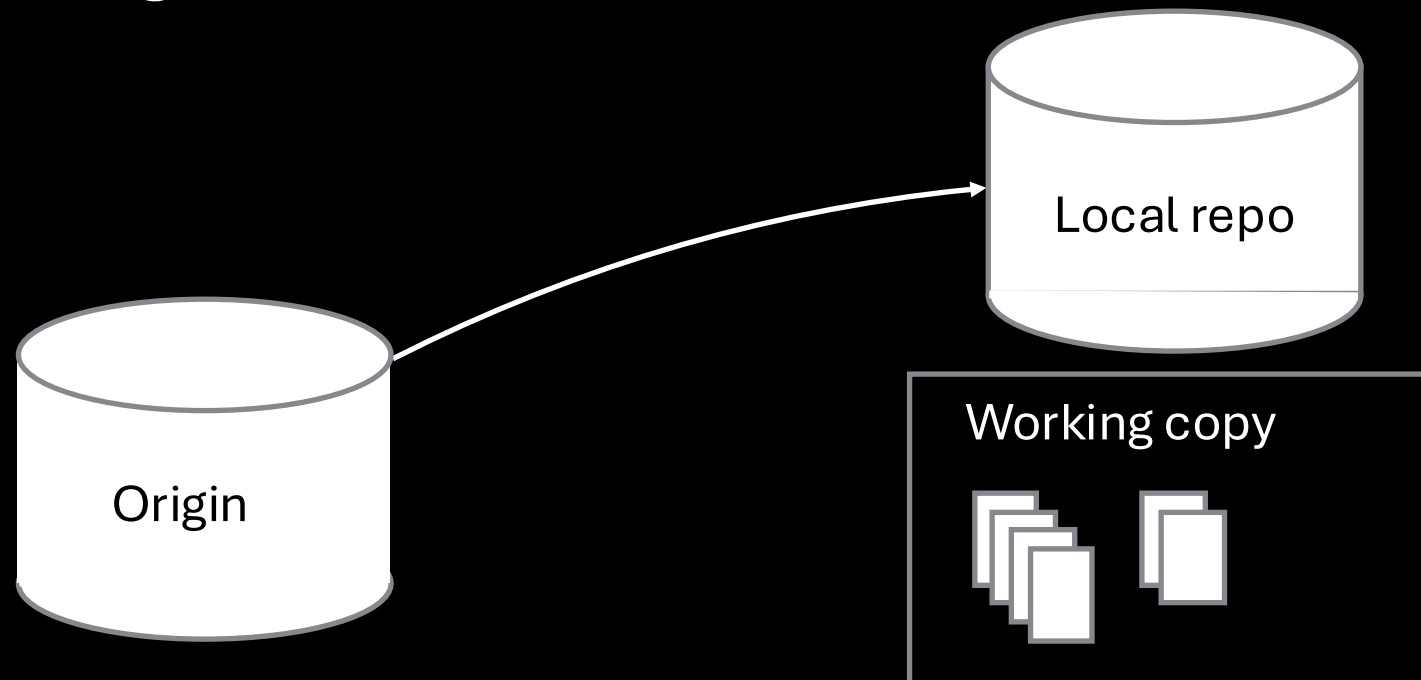
Clone to get local repo



- The “clone” command makes a copy of the remote repository on your local machine.
- Now there are two repositories.

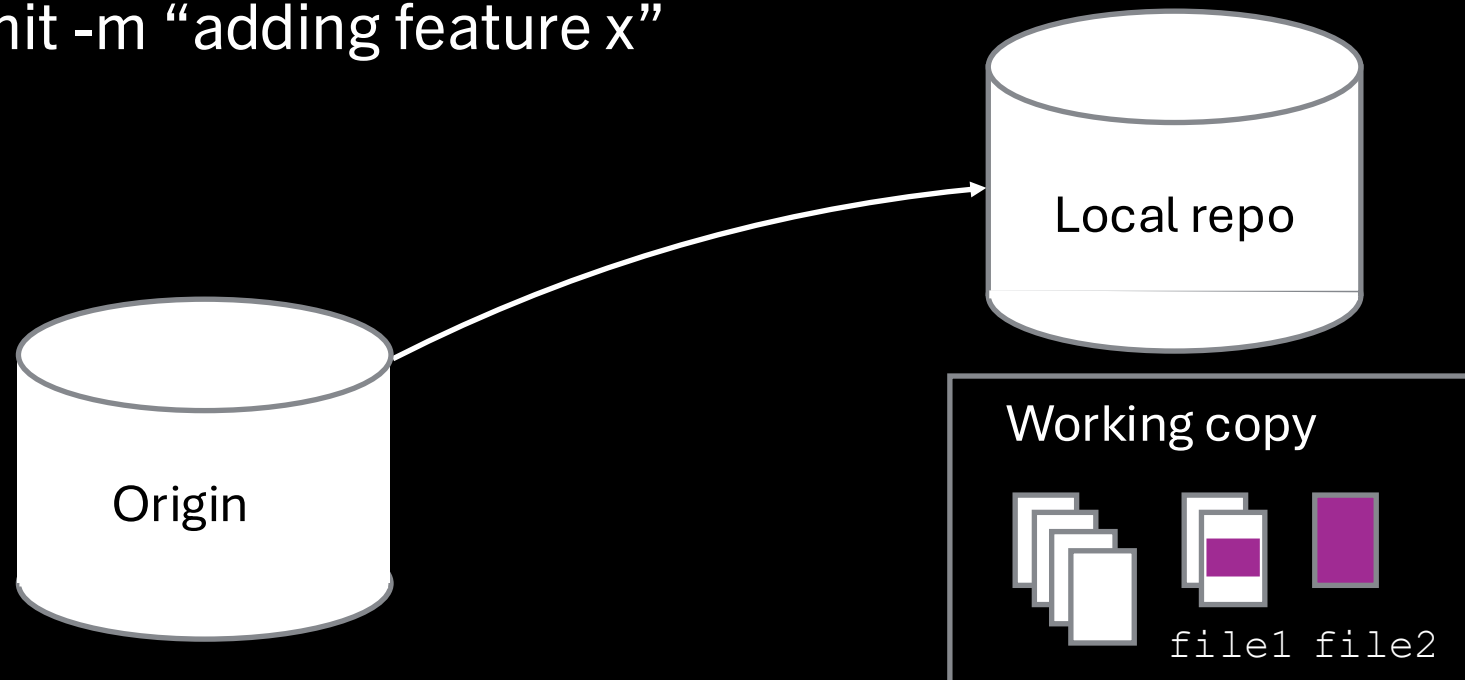
Local repository

- The actual repository is hidden (.git folder). What you see and edit is the working copy of the files from the repository.
- You can create new files, make changes to files, and delete files (as you usually do).
- When you want to “commit” a change to the local repository, you need to first “stage” the changes.



How to get work done

- Make changes to files, add new files.
- When you are ready to commit your work to your local repo, you need to tell git which files have changes that you want to commit.
- `git add file1 file2`
- `git commit -m "adding feature x"`



Staging changes

- `git add` doesn't add files to your repo. Instead, it marks a file as being part of the current change.
- This means that when you make some changes to a file and then add and commit them, the next time you make some changes to a file you will still have to run `git add` to add the changes to the next commit.

git status

- A file can be in one of 4 states:
 - **untracked** – you have never run a git command on the file
 - **tracked** – committed
 - **staged** – `git add` has been used to add changes in this file to the next commit
 - **dirty/modified** – the file has changes that haven't been staged

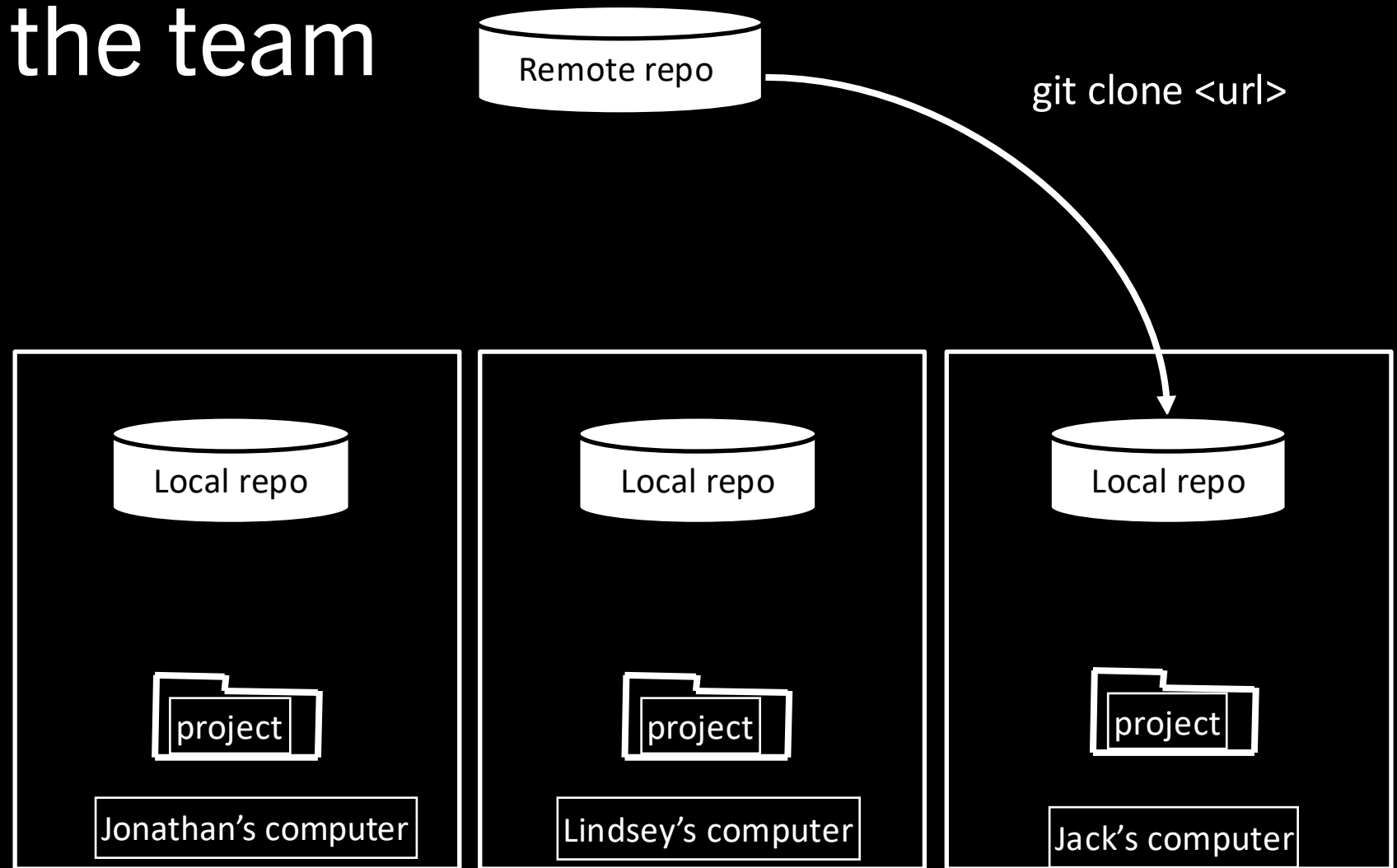
TIP: Use `git status` *regularly*. It helps you make sure the changes you have made really make it into the repo!

- IntelliJ uses colours and symbols to help you easily see the status of files

Basic workflow (No Branching)

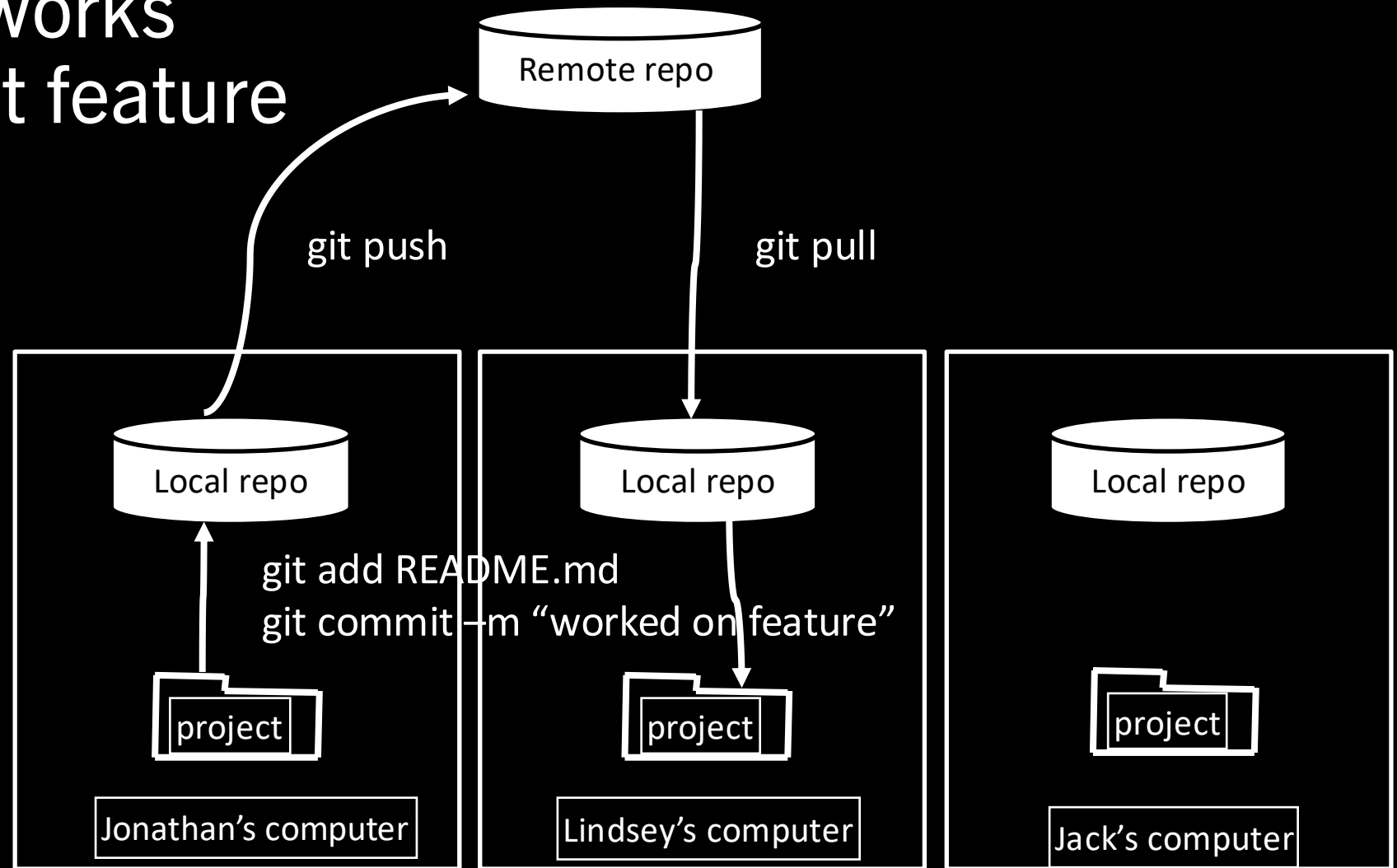
- Starting a project:
 - `git clone <url>`
- Normal work:
 - After you have made some changes
 - `git status` (see what has really changed)
 - `git add file1 file2 file3`
 - `git commit -m "meaningful commit message"`
 - `git push`
- In IntelliJ, you can either type these commands in the Terminal or use the GUI that IntelliJ provides.

Distributed Version Control Systems: Jack joins the team



Distributed Version Control Systems:

JONATHAN works
on his current feature



Distributed Version Control Systems: Jack has a conflict

A **conflict** can occur between two versions of the same file.

Everywhere they differ, both versions are shown.

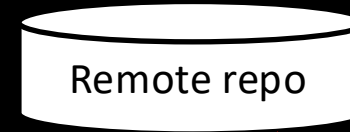
To resolve it, choose which edits to keep and add/commit/push these changes.

Example diff for README.md

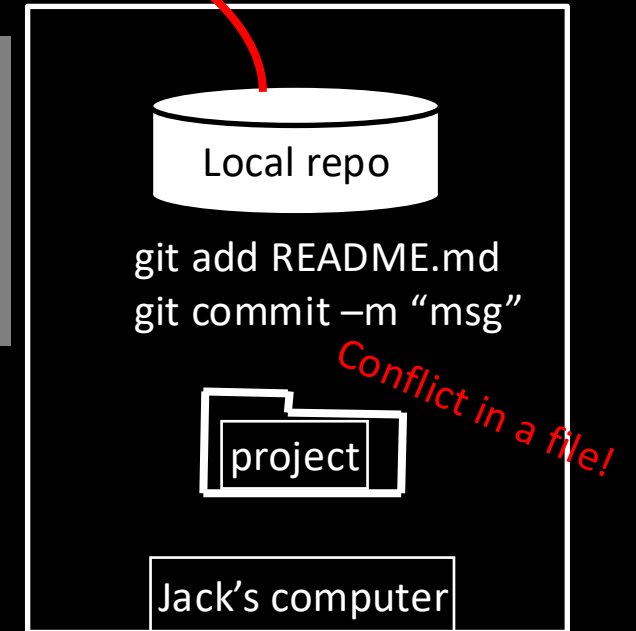
```
<<<<<< HEAD
# Project X - The ultimate developer toolkit
=====
# Project X - A simple developer utility
>>>>>> origin/main
```

README.md with conflict resolved

```
# Project X - The ultimate developer utility toolkit
```



git push ← this will initially fail due to the conflict and tell Jack to pull and manually resolve any conflicts before pushing



Branch and Merge Workflow

- To avoid having to constantly resolve conflicts, developers often create a **branch**, where they work on a new feature, then submit a **pull request**.
- One or more members of the team review the pull request, resolve any conflicts (if needed), and the branch is merged back in.
- You'll get lots of practice with this in your project this term and this workflow will be more formally covered in labs and the course notes.
- If you are interested, you can also read about other workflows here:

<https://www.atlassian.com/git/tutorials/comparing-workflows>

THIS IS GIT. IT TRACKS COLLABORATIVE WORK
ON PROJECTS THROUGH A BEAUTIFUL
DISTRIBUTED GRAPH THEORY TREE MODEL.

COOL. HOW DO WE USE IT?

NO IDEA. JUST MEMORIZE THESE SHELL
COMMANDS AND TYPE THEM TO SYNC UP.
IF YOU GET ERRORS, SAVE YOUR WORK
ELSEWHERE, DELETE THE PROJECT,
AND DOWNLOAD A FRESH COPY.



Extra Git Resources

What is version control and overview of commands:

<https://betterexplained.com/articles/a-visual-guide-to-version-control/>

A quick guide to getting started using git:

<https://towardsdatascience.com/a-quick-primer-to-version-control-using-git-3fbdbb123262>

A custom version of <https://learngitbranching.js.org> will also be used in this course for some online exercises. This will be announced and linked from Quercus once available.